Experiment No. 1: 8-Puzzle Problem

AIM

To write a Python program to solve the 8-Puzzle problem using the A* search algorithm with the Manhattan-distance heuristic, and to display the sequence of moves from the initial state to the goal state.

ALGORITHM

1. Represent a state as a tuple of 9 numbers (row by row); 0 denotes the blank. Define the goal state (1 2 3 / 4 5 6 / 7 8 _).
2. Check solvability: count the inversions among the non-blank tiles; the puzzle is solvable only if the count is even.
3. Put the start state in the open list (priority queue) with f(n) = g(n) + h(n), where g is the number of moves made and h is the sum of Manhattan distances of all tiles from their goal positions.
4. Remove the state with the smallest f from the open list. If it is the goal, trace the parent links back to the start to obtain the solution path and stop.
5. Otherwise generate the successors by sliding the blank Up, Down, Left or Right (only legal moves); each successor has g + 1.
6. If a successor is new, or has been reached with a smaller g than before, record its parent and g and insert it into the open list with its f value.
7. Repeat steps 4 to 6 until the goal is found; print the moves and the number of nodes expanded.

PROGRAM

```
# 8-Puzzle problem solved with A* search (Manhattan-distance heuristic)
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)          # 0 represents the blank tile


def manhattan(state):
    """Sum of horizontal + vertical distances of every tile from its goal cell."""
    dist = 0
    for i, tile in enumerate(state):
        if tile:
            g = GOAL.index(tile)
            dist += abs(i // 3 - g // 3) + abs(i % 3 - g % 3)
    return dist


def is_solvable(state):
    """A 3x3 puzzle is solvable only if the number of inversions is even."""
    tiles = [t for t in state if t]
    inv = sum(1 for i in range(len(tiles))
              for j in range(i + 1, len(tiles)) if tiles[i] > tiles[j])
    return inv % 2 == 0


def neighbours(state):
    """Generate (move, new_state) pairs by sliding the blank."""
    z = state.index(0)
    r, c = divmod(z, 3)
    for move, dr, dc in (("Up", -1, 0), ("Down", 1, 0), ("Left", 0, -1), ("Right", 0, 1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            n = nr * 3 + nc
            s = list(state)
            s[z], s[n] = s[n], s[z]
            yield move, tuple(s)
def solve(start):
```

```
    open_list = [(manhattan(start), 0, start)]        # (f, g, state)
    parent = {start: (None, None)}
    best_g = {start: 0}
    expanded = 0
    while open_list:
        f, g, state = heapq.heappop(open_list)
        if state == GOAL:
            path = []
            while parent[state][0] is not None:
                path.append((parent[state][1], state))
                state = parent[state][0]
            return path[::-1], expanded
        if g > best_g[state]:
            continue
        expanded += 1
        for move, nxt in neighbours(state):
            ng = g + 1
            if nxt not in best_g or ng < best_g[nxt]:
                best_g[nxt] = ng
                parent[nxt] = (state, move)
                heapq.heappush(open_list, (ng + manhattan(nxt), ng, nxt))
    return None, expanded

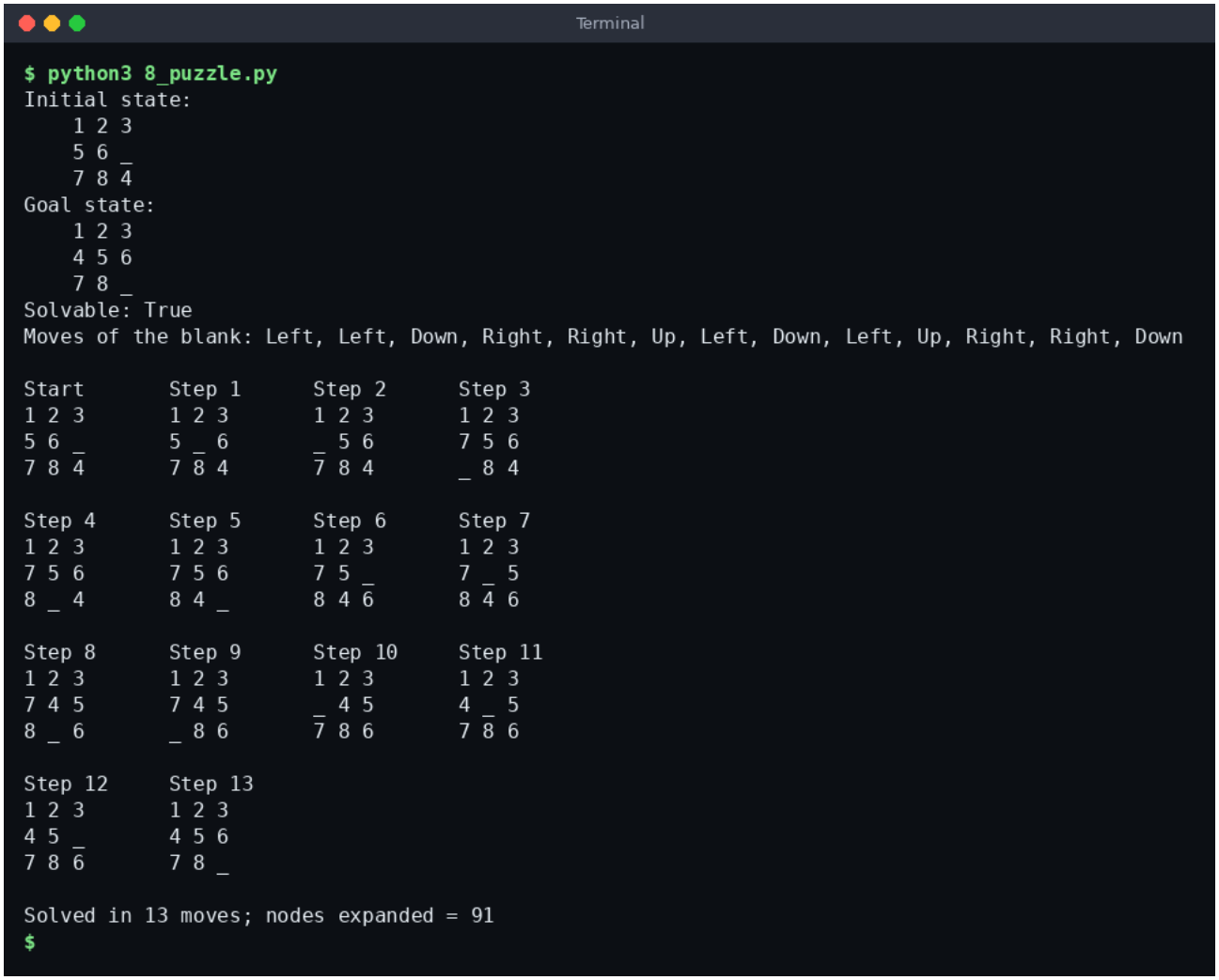

def show(state):
    for r in range(3):
        print("   ", " ".join(str(x) if x else "_" for x in state[3 * r:3 * r + 3]))


def show_row(frames, per_row=4):
    """Print several boards side by side; frames = list of (label, state)."""
    for k in range(0, len(frames), per_row):
        chunk = frames[k:k + per_row]
        print("   ".join(f"{label:<9}" for label, _ in chunk))
        for r in range(3):
            print("   ".join(f"{' '.join(str(x) if x else '_' for x in s[3 * r:3 * r + 3]):<9}"
                             for _, s in chunk))
        print()


start = (1, 2, 3, 5, 6, 0, 7, 8, 4)
print("Initial state:")
show(start)
print("Goal state:")
show(GOAL)
print("Solvable:", is_solvable(start))

path, expanded = solve(start)
print("Moves of the blank:", ", ".join(move for move, _ in path))
print()
show_row([("Start", start)] + [(f"Step {i}", st) for i, (_, st) in enumerate(path, 1)])
print(f"Solved in {len(path)} moves; nodes expanded = {expanded}")
```

OUTPUT



RESULT

Thus the 8-Puzzle problem was solved using A* search and the program executed successfully.

Experiment No. 2: 8-Queens Problem

AIM

To write a Python program to place 8 queens on an 8×8 chessboard so that no two queens attack each other, using the backtracking technique, and to count all possible solutions.

ALGORITHM

1. Place the queens column by column; the list rows holds the row index of the queen in each column filled so far.
2. For the current column, try each row 0 to 7 in turn.
3. A position is safe if no earlier queen is in the same row and no earlier queen is on the same diagonal (|row difference| = |column difference|).
4. If the position is safe, place the queen there and recursively place the queens in the next column.
5. If all 8 columns are filled, store the arrangement as a solution.
6. If no row is safe in a column, backtrack: remove the previous queen and try its next row.
7. Continue until all possibilities are explored; print the number of solutions and display the first solution as a board.

PROGRAM

```
# 8-Queens problem solved with backtracking
N = 8

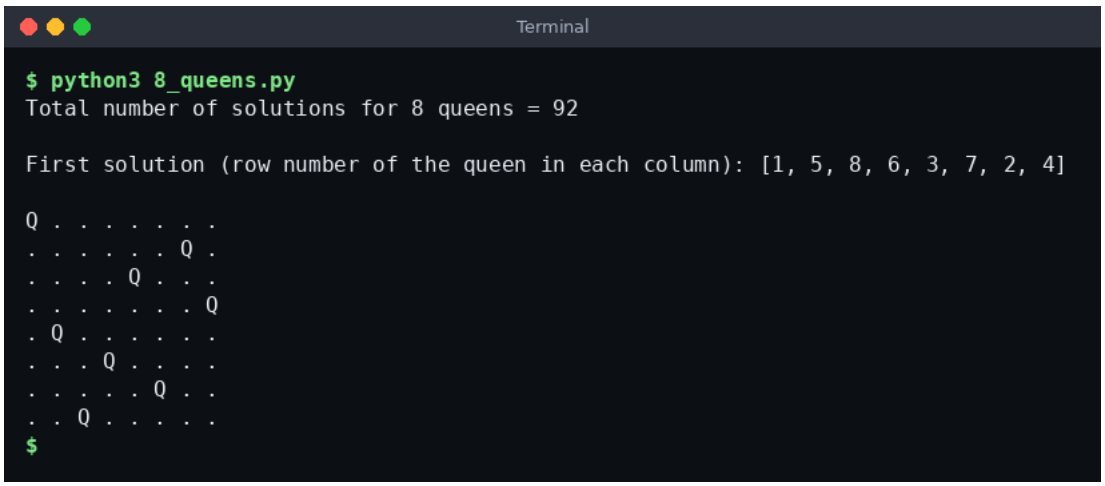

def is_safe(rows, row):
    """rows[c] is the row of the queen already placed in column c."""
    col = len(rows)
    for c, r in enumerate(rows):
        if r == row or abs(r - row) == abs(c - col):   # same row or same diagonal
            return False
    return True


def place(rows, solutions):
    if len(rows) == N:                  # all columns filled
        solutions.append(rows[:])
        return
    for row in range(N):
        if is_safe(rows, row):
            rows.append(row)            # place queen
            place(rows, solutions)      # go to next column
            rows.pop()                  # backtrack


solutions = []
place([], solutions)
print("Total number of solutions for", N, "queens =", len(solutions))

first = solutions[0]
print("\nFirst solution (row number of the queen in each column):", [r + 1 for r in first])
print()
for r in range(N):
    print(" ".join("Q" if first[c] == r else "." for c in range(N)))
```

OUTPUT



RESULT

Thus the 8-Queens problem was solved using backtracking; 92 solutions were found and the program executed successfully.

Experiment No. 3: Water Jug Problem

AIM

To write a Python program to measure exactly the required amount of water using two jugs of given capacities (4 litres and 3 litres, target 2 litres) using Breadth-First Search.

ALGORITHM

1. Represent a state as (a, b), the litres of water in jug A and jug B. The initial state is (0, 0).
2. A target is reachable only if it is not larger than the bigger jug and is a multiple of gcd(capacity A, capacity B); otherwise report that no solution exists.
3. Insert the initial state in a queue and mark it visited.
4. Remove the first state from the queue; if either jug contains the target amount, trace back through the parent links and stop.
5. Generate the next states using the six operations: fill A, fill B, empty A, empty B, pour A into B (until B is full or A is empty), pour B into A.
6. Add every state that has not been visited to the queue and remember its parent state and operation.
7. Repeat steps 4 to 6 until the goal is found; BFS therefore gives the shortest sequence of operations.

PROGRAM

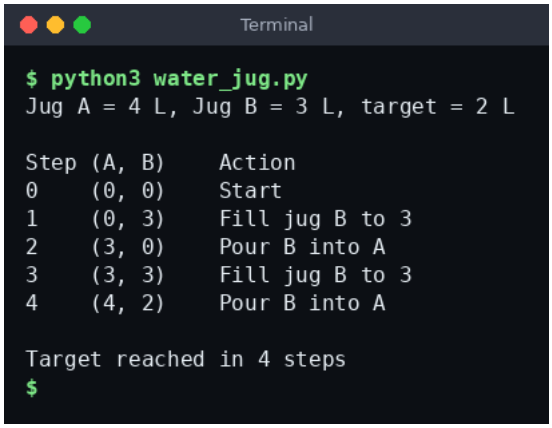
```
# Water Jug problem solved with Breadth-First Search
from collections import deque
from math import gcd


def water_jug(cap_a, cap_b, target):
    if target > max(cap_a, cap_b) or target % gcd(cap_a, cap_b) != 0:
        return None                                     # no solution possible
    start = (0, 0)
    parent = {start: None}
    queue = deque([start])
    while queue:
        a, b = queue.popleft()
        if a == target or b == target:                  # goal test
            path = []
            state = (a, b)
            while state is not None:
                path.append((state, parent[state][1] if parent[state] else "Start"))
                state = parent[state][0] if parent[state] else None
            return path[::-1]
        pour_ab = min(a, cap_b - b)                     # amount poured from A to B
        pour_ba = min(b, cap_a - a)                     # amount poured from B to A
        moves = [
            ((cap_a, b), f"Fill jug A to {cap_a}"),
            ((a, cap_b), f"Fill jug B to {cap_b}"),
            ((0, b), "Empty jug A"),
            ((a, 0), "Empty jug B"),
            ((a - pour_ab, b + pour_ab), "Pour A into B"),
            ((a + pour_ba, b - pour_ba), "Pour B into A"),
        ]
        for nxt, action in moves:
            if nxt not in parent:
                parent[nxt] = ((a, b), action)
                queue.append(nxt)
    return None


cap_a, cap_b, target = 4, 3, 2
print(f"Jug A = {cap_a} L, Jug B = {cap_b} L, target = {target} L\n")
path = water_jug(cap_a, cap_b, target)
if path is None:
    print("No solution exists")
else:
```

```
    print(f"{'Step':<5}{'(A, B)':<10}Action")
    for i, (state, action) in enumerate(path):
        print(f"{i:<5}{str(state):<10}{action}")
    print(f"\nTarget reached in {len(path) - 1} steps")
```

OUTPUT



RESULT

Thus the Water Jug problem was solved using Breadth-First Search and the program executed successfully.

Experiment No. 4: Cryptarithmetic Problem

AIM

To write a Python program to solve cryptarithmetic puzzles such as SEND + MORE = MONEY and TWO + TWO = FOUR by assigning a distinct digit to every letter.

ALGORITHM

1. Read the two addend words and the result word; collect the set of distinct letters (at most 10).
2. Note the leading letters of the three words; they must not be assigned the digit 0.
3. Generate every assignment of distinct digits 0 to 9 to the letters (permutations).
4. Reject any assignment in which a leading letter is 0.
5. Convert each word into a number using the assignment and test whether word1 + word2 = result.
6. If the equation holds, the assignment is a solution; print the letter mapping and the numerical check.
7. Continue (for puzzles with several answers) until all permutations are tested and count the solutions.

PROGRAM

```
# Cryptarithmetic puzzle solver (generate-and-test with pruning of leading zeros)
from itertools import permutations


def solve(word1, word2, result):
    letters = sorted(set(word1 + word2 + result))
    if len(letters) > 10:
        return
    leading = {word1[0], word2[0], result[0]}

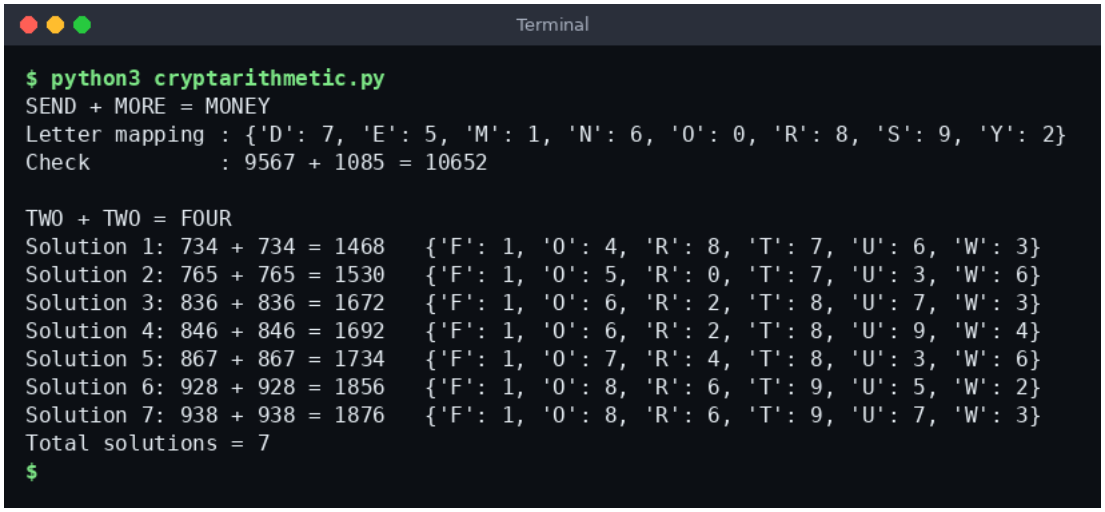
    def value(word, m):
        n = 0
        for ch in word:
            n = n * 10 + m[ch]
        return n

    for digits in permutations(range(10), len(letters)):   # every distinct digit assignment
        m = dict(zip(letters, digits))
        if any(m[ch] == 0 for ch in leading):               # leading letters cannot be 0
            continue
        if value(word1, m) + value(word2, m) == value(result, m):
            yield m, value(word1, m), value(word2, m), value(result, m)


# Puzzle 1 : SEND + MORE = MONEY
m, a, b, c = next(solve("SEND", "MORE", "MONEY"))
print("SEND + MORE = MONEY")
print("Letter mapping :", dict(sorted(m.items())))
print(f"Check          : {a} + {b} = {c}")

# Puzzle 2 : TWO + TWO = FOUR (all solutions)
print("\nTWO + TWO = FOUR")
count = 0
for m, a, b, c in solve("TWO", "TWO", "FOUR"):
    count += 1
    print(f"Solution {count}: {a} + {b} = {c}   {dict(sorted(m.items()))}")
print("Total solutions =", count)
```

OUTPUT



RESULT

Thus the cryptarithmetic puzzles were solved and the program executed successfully.

Experiment No. 5: Missionaries and Cannibals Problem

AIM

To write a Python program to move 3 missionaries and 3 cannibals across a river using a boat that carries at most two people, so that missionaries are never outnumbered by cannibals on either bank, using Breadth-First Search.

ALGORITHM

1. Represent a state as (M, C, B): missionaries and cannibals on the left bank and the boat side (0 = left, 1 = right). Start = (3, 3, 0), goal = (0, 0, 1).
2. Define the possible boat loads: (1,0), (0,1), (2,0), (0,2), (1,1) missionaries and cannibals.
3. A state is safe if, on each bank, the missionaries are either absent or at least as many as the cannibals.
4. Insert the start state in a queue. Remove the first state; if it is the goal, trace the parents back and stop.
5. Generate successors by moving a boat load from the bank where the boat is; discard states with invalid counts or states that are not safe.
6. Add each unvisited successor to the queue and remember its parent and the move used.
7. Repeat until the goal is reached; BFS gives the minimum number of crossings. Print each crossing.

PROGRAM

```
# Missionaries and Cannibals problem solved with Breadth-First Search
from collections import deque

# State = (missionaries on left, cannibals on left, boat side: 0 = left, 1 = right)
START, GOAL = (3, 3, 0), (0, 0, 1)
MOVES = [(1, 0), (0, 1), (2, 0), (0, 2), (1, 1)]          # (missionaries, cannibals) in boat


def is_safe(m, c):
    """Missionaries must never be outnumbered on either bank (unless there are none)."""
    left_ok = (m == 0 or m >= c)
    right_ok = (3 - m == 0 or 3 - m >= 3 - c)
    return left_ok and right_ok

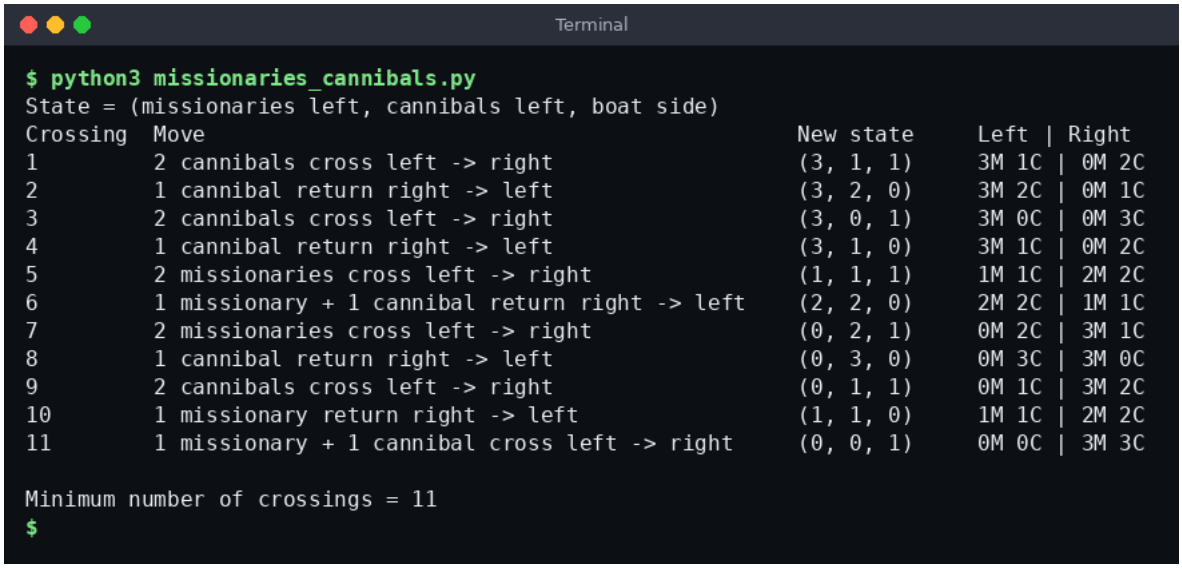

def successors(state):
    m, c, boat = state
    for dm, dc in MOVES:
        nm, nc = (m - dm, c - dc) if boat == 0 else (m + dm, c + dc)
        if 0 <= nm <= 3 and 0 <= nc <= 3 and is_safe(nm, nc):
            yield (dm, dc), (nm, nc, 1 - boat)


def bfs():
    parent = {START: None}
    queue = deque([START])
    while queue:
        state = queue.popleft()
        if state == GOAL:
            path = []
            while parent[state] is not None:
                prev, move = parent[state]
                path.append((move, state))
                state = prev
            return path[::-1]
        for move, nxt in successors(state):
            if nxt not in parent:
                parent[nxt] = (state, move)
                queue.append(nxt)
def describe(prev, move):
    dm, dc = move
```

```
    people = []
    if dm:
        people.append(f"{dm} missionary" if dm == 1 else f"{dm} missionaries")
    if dc:
        people.append(f"{dc} cannibal" if dc == 1 else f"{dc} cannibals")
    return " + ".join(people) + (" cross left -> right" if prev[2] == 0 else " return right -> left")


path = bfs()
print("State = (missionaries left, cannibals left, boat side)")
print(f"{'Crossing':<10}{'Move':<50}{'New state':<14}Left | Right")
state = START
for i, (move, nxt) in enumerate(path, 1):
    print(f"{i:<10}{describe(state, move):<50}{str(nxt):<14}"
          f"{nxt[0]}M {nxt[1]}C | {3 - nxt[0]}M {3 - nxt[1]}C")
    state = nxt
print(f"\nMinimum number of crossings = {len(path)}")
```

OUTPUT



RESULT

Thus the Missionaries and Cannibals problem was solved using BFS (minimum 11 crossings) and the program executed successfully.